

Competitive Programming

Section 30 Editorial: GCD, LCM, and Diophantine Equations

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial map	4
2	Shared tool: extended Euclid	5
2.1	Shared C++ helper	5
2.2	Shared Python helper	5
3	A. GCD LCM Queries	6
3.1	Restatement and traps	6
3.2	Derivation and pseudocode	6
3.3	Worked trace	6
3.4	Correctness proof	7
3.5	Complexity	7
3.6	C++ and Python notes	7
3.7	Wrong answers and targeted tests	7
4	B. General Inverse	8
4.1	Restatement and traps	8
4.2	Derivation and pseudocode	8
4.3	Worked traces	8
4.4	Correctness proof	9
4.5	Complexity	9
4.6	C++ and Python notes	9
4.7	Wrong answers and targeted tests	9
5	C. Linear Diophantine	10
5.1	Restatement and traps	10
5.2	Derivation and pseudocode	10
5.3	Worked traces	10
5.4	Correctness proof	11
5.5	Complexity	11
5.6	C++ and Python notes	11
5.7	Wrong answers and targeted tests	11
6	Formative upsolve exercises	12
7	Local workflow	13
8	Carry-forward checklist	14
9	Full reference implementations	15
9.1	A. GCD LCM Queries: C++	15
9.2	A. GCD LCM Queries: Python	15
9.3	B. General Inverse: C++	16
9.4	B. General Inverse: Python	17
9.5	C. Linear Diophantine: C++	17

9.6	C. Linear Diophantine: Python	18
-----	---	----

1 Editorial map

Problem	Constraints that choose the method	Core result
A. GCD LCM Queries	$q \leq 2 \cdot 10^5$, positive a, b $\leq 10^9$	Euclid, then $(a/g) \cdot b$
B. General Inverse	$q \leq 2 \cdot 10^5$, arbitrary positive $m \leq 10^9$	extended Euclid, not Fermat
C. Linear Diophantine	$q \leq 2 \cdot 10^5$, positive a, b, c $\leq 10^9$	gcd divisibility, then Bezout scaling

All three problems are designed for $O(\log \text{ value})$ work per query. A linear search for a divisor, inverse, or coefficient pair is not competitive at these limits.

2 Shared tool: extended Euclid

Use this contract throughout the section:

```
extended_gcd(a,b) returns (g,x,y) such that  
g = gcd(a,b) and  $a*x + b*y = g$ .
```

The recursive transition is:

```
if b == 0: return (a,1,0)  
(g,x1,y1) = extended_gcd(b,a mod b)  
return (g,y1,x1-floor(a/b)*y1)
```

The last line is not a memorized trick. It follows from replacing $a \bmod b$ by $a - \text{floor}(a/b)*b$ in the recursive Bezout identity. Re-derive it whenever you are unsure which coefficient belongs to a .

2.1 Shared C++ helper

```
long long egcd(long long a, long long b, long long &x, long long &y) {  
    if (b == 0) {  
        x = 1;  
        y = 0;  
        return a;  
    }  
    long long x1, y1;  
    long long g = egcd(b, a % b, x1, y1);  
    x = y1;  
    y = x1 - (a / b) * y1;  
    return g;  
}
```

2.2 Shared Python helper

```
def egcd(a: int, b: int) -> tuple[int, int, int]:  
    if b == 0:  
        return a, 1, 0  
    g, x1, y1 = egcd(b, a % b)  
    return g, y1, x1 - (a // b) * y1
```

For this section's positive inputs both versions return a positive gcd. In a negative-input variant, define and enforce a positive-gcd contract before reusing this code.

3 A. GCD LCM Queries

3.1 Restatement and traps

For every positive pair (a,b) , print:

```
gcd(a,b) lcm(a,b)
```

The trap is intermediate overflow. Although the local maximum lcm is at most 10^{18} and fits in signed `long long`, $a*b$ must not be formed before removing the gcd. Also do not try to enumerate divisors up to $\sqrt{a}$ for every query: $2*10^5$ queries make that approach too slow.

3.2 Derivation and pseudocode

Euclid preserves common divisors under $(a,b) \rightarrow (b, a \bmod b)$. Once the second entry is zero, the first is the gcd. The prime-exponent identity gives:

```
lcm(a,b) = a/gcd(a,b) * b
```

Pseudocode:

```
for each (a,b):  
    g = gcd(a,b)  
    l = (a/g)*b  
    print g, l
```

3.3 Worked trace

For $a=84$, $b=126$:

```
gcd(84,126) = gcd(126,84) = gcd(84,42) = gcd(42,0) = 42  
lcm = (84/42)*126 = 252
```

The result passes two useful checks: both inputs divide 252, and $42*252 = 84*126$.

3.4 Correctness proof

Euclid's recurrence preserves the set of common divisors because $a \bmod b = a - qb$; hence it returns $\gcd(a, b)$. For each prime, $\gcd$ takes the smaller input exponent and lcm takes the larger one. Their exponent sum equals the exponent in $a * b$, proving $\gcd(a, b) * \text{lcm}(a, b) = a * b$. Therefore $(a/g) * b$ equals the lcm, and the algorithm prints both required values. $\square$

3.5 Complexity

Each $\gcd$ takes $O(\log \min(a, b))$; lcm arithmetic is constant time. Total time is $O(q \log \max(a, b))$ and extra memory is $O(1)$.

3.6 C++ and Python notes

In C++, `std::gcd(a, b)` and `long long l = a / g * b;` are enough under the local bounds. Python uses `math.gcd` and has arbitrary precision, but the same operation order documents the intended safe arithmetic.

```
long long g = gcd(a, b);  
long long l = a / g * b;
```

```
g = math.gcd(a, b)  
l = a // g * b
```

3.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
$a * b / g$ in narrow arithmetic	large coprime values	multiplication can overflow first
output $a * b$ as lcm	12 18	common factors are counted twice
trial-divide every query	many near- 10^9 queries	query total is too large

Test equal numbers ($10^9, 10^9$), coprime numbers (7, 5), and one number dividing the other (9, 45).

4 B. General Inverse

4.1 Restatement and traps

For each positive pair (a,m) , print the smallest nonnegative x satisfying:

$$a \cdot x \equiv 1 \pmod{m}$$

or -1 when no inverse exists. The modulus is not guaranteed prime, so Fermat's a^{m-2} formula is invalid in general. The local contract explicitly uses 0 when $m=1$.

4.2 Derivation and pseudocode

An inverse exists precisely when there are integers x,y such that:

$$a \cdot x + m \cdot y = 1$$

Extended Euclid creates that equation exactly when $\gcd(a,m)=1$.

```
for each (a,m):
    if m == 1:
        print 0
        continue
    (g,x,y) = extended_gcd(a,m)
    if g != 1:
        print -1
    else:
        print normalized residue of x modulo m
```

For C++, normalize with $(x \% m + m) \% m$ or the two-step version from the lesson. Python's $x \% m$ is already nonnegative for positive m .

4.3 Worked traces

4.3.1 Inverse exists: $a=10, m=17$

Back-substitution gives:

$$1 = 2 \cdot 17 - 3 \cdot 10$$

Thus $x=-3$. Normalize: $-3 \bmod 17 = 14$. Check $10 \cdot 14 = 140 \equiv 1 \pmod{17}$.

4.3.2 Inverse does not exist: $a=6, m=9$

$\text{gcd}(6,9)=3$, so every value $6x+9y$ is divisible by three. It cannot equal one; print -1 without searching residues.

4.4 Correctness proof

If the algorithm prints a normalized coefficient x , extended Euclid gave $a*x+m*y=1$; reducing modulo m proves $a*x \equiv 1 \pmod{m}$. Conversely, if an inverse exists, rearranging its congruence gives an equation $a*x+m*y=1$. Bezout's identity then requires $\text{gcd}(a,m)=1$. Thus the gcd test rejects exactly the impossible cases and the returned coefficient is exactly the required inverse. $\square$

4.5 Complexity

Extended Euclid takes $O(\log \min(a,m))$ time and $O(\log \min(a,m))$ recursion depth, with $O(1)$ auxiliary data at each level. Iterative implementations can make the stack use $O(1)$.

4.6 C++ and Python notes

Do not call $\text{pow}(a,m-2,m)$ unless the statement establishes a prime modulus. In C++, retain the signed x from egcd until after the gcd check. In Python, the tuple helper directly returns (g,x,y) .

```
long long g = egcd(a, m, x, y);
if (g != 1) cout << -1;
else cout << (x % m + m) % m;
```

```
g, x, _ = egcd(a, m)
answer = -1 if g != 1 else x % m
```

4.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
use Fermat for every modulus	6 35	35 is composite, though inverse exists
print raw coefficient	10 17	-3 is valid but not the requested representative
return an inverse when gcd is not one	6 9	no such residue exists
omit local modulus-one branch	25 1	required output is 0

Test a coprime pair with a negative Bezout coefficient, a shared-factor pair, $a=1$, and $m=1$.

5 C. Linear Diophantine

5.1 Restatement and traps

For each positive triple (a,b,c) , print any integers (x,y) satisfying:

$$a \cdot x + b \cdot y = c$$

or print IMPOSSIBLE. The variables are integers, not nonnegative integers; negative answers are often necessary. Do not divide c by the gcd until after checking that the division is exact.

5.2 Derivation and pseudocode

Let $(g,x_0,y_0) = \text{extended_gcd}(a,b)$. The equation is solvable iff $c \% g == 0$. When that condition holds, scale the Bezout equation by c/g .

```
for each (a,b,c):
    (g,x0,y0) = extended_gcd(a,b)
    if c mod g != 0:
        print IMPOSSIBLE
    else:
        scale = c/g
        print x0*scale, y0*scale
```

5.3 Worked traces

5.3.1 Solvable: $6x + 10y = 14$

Extended Euclid gives $6 \cdot 2 + 10 \cdot (-1) = 2$. Since $14/2=7$, scale the pair:

$$\begin{aligned} x &= 14, \quad y = -7 \\ 6 \cdot 14 + 10 \cdot (-7) &= 14 \end{aligned}$$

5.3.2 Impossible: $6x + 10y = 15$

The left side is always divisible by two, while fifteen is not. The gcd test therefore rejects the equation before any coefficient is printed.

5.4 Correctness proof

Every integer combination $a*x+b*y$ is divisible by $g=\gcd(a,b)$, so if g does not divide c , no solution exists. If it does divide c , extended Euclid returns $a*x_0+b*y_0=g$; multiplying by c/g gives:

$$a*(x_0*c/g) + b*(y_0*c/g) = c.$$

Thus the algorithm prints IMPOSSIBLE exactly when necessary and otherwise prints a valid integer solution. $\square$

5.5 Complexity

One extended-gcd call costs $O(\log \min(a,b))$ per query. Scaling and printing are constant time; total extra memory is the recursion stack.

5.6 C++ and Python notes

Use `long long` for the local bounds. The local construction can produce negative values, so do not use unsigned types. In a higher-bound version, cast the scale multiplication to `__int128` in C++, and decide how the output contract handles values outside signed 64-bit range.

```
long long g = egcd(a, b, x, y);
if (c % g != 0) cout << "IMPOSSIBLE\n";
else cout << x * (c / g) << ' ' << y * (c / g) << '\n';
```

```
g, x, y = egcd(a, b)
if c % g:
    print("IMPOSSIBLE")
else:
    scale = c // g
    print(x * scale, y * scale)
```

5.7 Wrong answers and targeted tests

Wrong approach	Input	Why it fails
require nonnegative x,y	6 10 14	a valid construction has $y=-7$
scale without divisibility check	6 10 15	$15/2$ is not an integer scale
solve by bounded brute force	coefficients near 10^9	no useful finite search range
swap returned coefficients	15 21 12	substituted equation exposes the mistake

Always test the exact equation after printing. Include a case with $\gcd=1$, a case where $\gcd$ does not divide c , and a case whose valid output has a negative coordinate.

6 Formative upsolve exercises

1. For A, explain why computing $(a/g)*b$ is safer than $a*b/g$, then evaluate the lcm of 1,000,000,000 and 1,000,000,000.

Hint: cancel the gcd before multiplying.

Answer: the gcd is 10^9 , so the lcm is 10^9 ; multiplying first is an unnecessary overflow risk in smaller integer types.

2. For B, use extended Euclid to find the inverse of 11 modulo 26.

Hint: $26 = 2*11 + 4$, then continue Euclid.

Answer: $11*19 = 209 = 1 \pmod{26}$, so the normalized inverse is 19.

3. For C, decide whether $35x + 15y = 12$ is solvable before constructing any coefficients.

Hint: compute only the gcd first.

Answer: it is **not** solvable because $\gcd(35,15)=5$ does not divide 12. This is the point of performing the divisibility test first.

7 Local workflow

Run the complete local suite with the reference target:

```
CP_TARGET=solution python3 sections/30_gcd_lcm_diophantine/check.py
```

Then run focused cases while developing:

```
A: (12,18), (7,5), (109,109)  
B: (3,11), (10,17), (6,9), (25,1)  
C: (6,10,14), (6,10,15), (7,5,1)
```

For C, a valid judge must check the equation itself rather than require one particular Bezout parameterization. Different valid pairs are expected.

8 Carry-forward checklist

1. Preserve a gcd or Bezout invariant; do not search candidates blindly.
2. Divide before multiplying for lcm, and still audit the final product range.
3. A modular inverse is a gcd-one statement, not automatically a Fermat statement.
4. For $ax+by=c$, gcd divisibility is both the impossibility test and the bridge to construction.
5. Treat negative coefficients as normal integer solutions, then verify by substitution.

9 Full reference implementations

9.1 A. GCD LCM Queries: C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b;
        cin >> a >> b;
        long long g = gcd(a, b);
        cout << g << ' ' << a / g * b << '\n';
    }
    return 0;
}
```

9.2 A. GCD LCM Queries: Python

```
import math
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b = data[idx], data[idx + 1]
        idx += 2
        g = math.gcd(a, b)
        out.append(f"{g} {a // g * b}")
    print("\n".join(out))
```

```
if __name__ == "__main__":
    main()
```

9.3 B. General Inverse: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, m;
        cin >> a >> m;
        if (m == 1) {
            cout << 0 << '\n';
            continue;
        }
        long long x, y;
        long long g = egcd(a, m, x, y);
        if (g != 1) {
            cout << -1 << '\n';
        } else {
            x %= m;
            if (x < 0) x += m;
            cout << x << '\n';
        }
    }
    return 0;
}
```

9.4 B. General Inverse: Python

```
import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, m = data[idx], data[idx + 1]
        idx += 2
        if m == 1:
            out.append("0")
            continue
        g, x, _ = egcd(a, m)
        out.append(str(x % m) if g == 1 else "-1")
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

9.5 C. Linear Diophantine: C++

```
#include <bits/stdc++.h>
using namespace std;

long long egcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

```

    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, c;
        cin >> a >> b >> c;
        long long x, y;
        long long g = egcd(a, b, x, y);
        if (c % g != 0) {
            cout << "IMPOSSIBLE\n";
        } else {
            long long scale = c / g;
            cout << x * scale << ' ' << y * scale << '\n';
        }
    }
    return 0;
}

```

9.6 C. Linear Diophantine: Python

```

import sys

def egcd(a: int, b: int) -> tuple[int, int, int]:
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b, c = data[idx], data[idx + 1], data[idx + 2]
        idx += 3
        g, x, y = egcd(a, b)
        if c % g != 0:
            out.append("IMPOSSIBLE")

```

```
        else:
            scale = c // g
            out.append(f"{x * scale} {y * scale}")
        print("\n".join(out))

if __name__ == "__main__":
    main()
```